Appendix: Artifact Description/Artifact Evaluation

Artifact Description (AD)

I. Overview of Contributions and Artifacts

A. Paper's Main Contributions

- C_1 We characterize the latency, bandwidth, and jitter sensitivity of AI communication workloads at scales up to 4,096 GPUs.
- C_2 We introduce a compositional linear-programming method with signature-based reuse for practical large-scale analysis.
- C_3 We use the results to derive network performance envelopes and identify useful provisioning points.

B. Computational Artifacts

The evaluation uses two artifacts:

- A_1 Code, input data, scripts, and results: https://github.com/tommasobo/SC26_ProvisioningAINetworks, branch main.
- A_2 Public execution traces: http://storage2.spcl.ethz.ch/traces/ai/.

Artifact	Contributions	Paper elements
A_1	C_1, C_2, C_3	Figs. 1, 3–10
A_2	C_1, C_2	Table II, Figs. 3–6

The requested badges are Artifacts Available, Artifacts Evaluated-Functional, and Results Reproduced. A persistent identifier will be provided before the freezing date (Stage 3, 25 AUG 2026).

II. Artifact Identification

A. Computational Artifact A_1

Relation To Contributions

Artifact A_1 contains the analysis implementation, plotting data, reproduction scripts, verification manifests, and final plots. The quick workflow generates Figures 1 and 3–10. Figure 2 is a method diagram. The full workflow freshly solves Figures 3–6 from trace-derived inputs and passes the generated CSV files to the paper-style plotters.

Expected Results

The quick workflow should produce nine PDF files under figures/. The full workflow should also produce result CSV files, logs, and task summaries in the selected scratch directory. The generated curves should follow the paper results.

The artifact assumes that execution traces have already been collected. It does not launch model training or trace collection.

Expected Reproduction Time (in Minutes)

Allow 15–30 minutes for installation. The quick workflow takes about one minute. Allow up to eight hours for the full workflow, depending on the machine, solver license availability, and worker count. Most individual analysis tasks should be allowed up to one hour. The optional Grok 4k latency and bandwidth analysis should be allowed approximately 3 to 5 days.

Artifact Setup (incl. Inputs)

Hardware: The quick workflow needs one CPU core, 2 GB of RAM, and about 100 MB of free space. For the full workflow, we recommend at least 4 CPU cores, 128 GB of RAM, and a scratch filesystem. No GPU is required. The optional 4,096-GPU Grok 4k analysis should use a large-memory node with at least 1 TB of RAM and sufficient scratch space.

Software: Python 3.10 or newer is required. Python packages are listed in requirements.txt; full verification also uses requirements-dev.txt and requirements-tierc.txt. The raw-trace path requires the nsys command from NVIDIA Nsight Systems. The full analysis requires Gurobi and a valid license. Academics can obtain a free Gurobi license from https://www.gurobi.com/academia/academic-program-and-licenses/. The LogGOPSim example additionally uses g++, gengetopt, and re2c. Slurm is optional.

Datasets / Inputs: The repository contains the numerical and processed inputs needed by the default workflows. Large trace and intermediate files should be kept on a scratch filesystem. Public traces are available from http://storage2.spcl.ethz.ch/traces/ai/. Input identities are recorded under local_artifact/manifests/.

Installation and Deployment:

```
git clone https://github.com/tommasobo/\
SC26_ProvisioningAINetworks.git
cd SC26_ProvisioningAINetworks
git checkout main
python3.11 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
```

For the full workflow, also install requirements-dev.txt and requirements-tierc.txt, then confirm that Gurobi can obtain a license.

Artifact Execution

Run the quick workflow locally:

```
./reproduce_quick.sh
```

Run the full workflow with a scratch directory:

```
./reproduce_full.sh \
  --scratch /scratch/$USER/provisioning_artifact \
  --workers 4
```

On Alps, submit the analysis through Slurm:

```
./reproduce_full.sh --slurm \
  --account <account> --partition normal \
  --scratch /iopsstor/scratch/cscs/$USER/artifact \
  --workers 4
```

The optional 4,096-GPU Grok 4k analysis is enabled only with –expensive_run and an explicit N1024 input directory. Before enabling it, confirm that the selected partition provides at least 1 TB of RAM and a multi-day wall-time limit.

Artifact Analysis (incl. Outputs)

The quick workflow writes the final PDFs under figures/. The full workflow writes one summary per task together with result CSV files and logs in the selected scratch directory.

B. Computational Artifact A_2

Relation To Contributions

Artifact A_2 contains the execution traces used to build communication graphs and exercise the analysis pipeline for Figures 3–6.

Expected Results

A selected trace should download with the recorded hash and pass through the trace-processing and simulation stages. The repository includes the input identities and reference results used by the final workflow.

Expected Reproduction Time (in Minutes)

Allow up to one hour for a selected trace download and replay. Processing a larger trace set can take several hours.

Artifact Setup (incl. Inputs)

Hardware: Use one CPU node with at least 32 GB of RAM and a scratch filesystem.

Software: The trace workflow uses wget, the nsys command, SQLite, the repository's Python tools, and Gurobi. LogGOPSim is used by the optional replay and Monolithic-LP stages.

Datasets / Inputs: The public traces are available at http://storage2.spcl.ethz.ch/traces/ai/. Filenames and SHA-256 hashes used by this artifact are recorded under local_artifact/manifests/.

Installation and Deployment: Install Artifact A_1, select one trace recorded in the manifests, and download it to scratch. Do not store large trace files in the repository or the home directory.

Artifact Execution

Run the selected trace workflow in a scratch directory. It exports the NSYS reports to SQLite, constructs the communication schedule and analysis inputs, runs the numerical analysis, and writes the result CSV and plot.

Artifact Analysis (incl. Outputs)

Confirm that the downloaded file's SHA-256 hash, the generated result, and the execution log are recorded in the output directory.

Artifact Evaluation (AE)

A. Computational Artifact A_1

Artifact Setup (incl. Inputs)

Clone branch main, create the Python environment, and install requirements.txt. For the full workflow, install requirements-dev.txt and requirements-tierc.txt, verify the Gurobi license, and choose a new scratch directory with sufficient free space. We recommend 4 CPU cores and 128 GB of RAM. On a cluster, use a compute allocation or the provided Slurm launcher.

Artifact Execution

Begin with the short installation and plotting check:

```
./reproduce_quick.sh
```

This verifies the Python environment and generates Figures 1 and 3–10 with the submission plotting scripts. Figure 2 is a method diagram.

Run the main numerical workflow with:

```
./reproduce_full.sh \
  --scratch /scratch/$USER/provisioning_artifact \
  --workers 4
```

The full workflow starts from the trace-derived collective and topology inputs. It builds the analysis problems, runs the solver and sensitivity sweeps for Figures 3–6, and writes fresh numerical results to scratch. It then passes those CSV files to the paper-style plotters. The generated PDFs use the same axes, labels, typography, and dimensions as the paper figures.

When raw NSYS reports are available, place them in the directory layout described in the repository README and add the trace root:

```
./reproduce_full.sh \
  --scratch /scratch/$USER/provisioning_artifact \
  --trace-root /scratch/$USER/raw_traces \
  --workers 4
```

This adds NSYS export, SQLite conversion, and communication-schedule generation before the numerical tasks. The subsequent solver and plotting steps use the outputs produced in that same run.

On a Slurm system, use the command from the AD section. The numerical tasks are submitted independently. Their job identifiers are written to submitted_jobs.txt. Wait for all jobs and the final plotting step to finish before inspecting the output. Do not enable –expensive_run during the standard evaluation.

Artifact Analysis (incl. Outputs)

A successful full run produces newly computed CSV files, one JSON manifest per analysis task, execution logs, and the final plot PDFs. Confirm that every task manifest reports status: ok and that Figures 1 and 3–10 are present under the scratch directory's figures/ subdirectory. The CSV files in scratch are the numerical inputs used by the final plotting step.

B. Computational Artifact A_2

Artifact Setup (incl. Inputs)

Install Artifact A_1 and prepare the selected raw NSYS reports using the directory layout described in the repository README. Keep the traces and all intermediate files in scratch.

Artifact Execution

Run the general raw-trace workflow:

```
./reproduce_full.sh \
  --scratch /scratch/$USER/provisioning_artifact \
  --trace-root /scratch/$USER/raw_traces \
  --workers 4
```

The workflow exports the NSYS reports to SQLite, constructs the GOAL schedules and analysis metadata, runs the numerical tasks, and passes the newly generated result CSV files to the paper-style plotting scripts. Use the Slurm option from the AD section when running on a cluster.

Artifact Analysis (incl. Outputs)

Confirm that the SQLite exports, GOAL schedules, analysis metadata, result CSV files, plots, manifests, and task logs were created in the selected scratch directory.